

04

SKT K-뉴딜 아카데미 · AI·
자동화 기초 · 9/29 오후

탐지 룰 ③

API 와 HTTP

실패가 아니라 성공을 의심하고,
회사 밖의 정보를 물어보는 법을 배웁니다.

경보를 세 줄로 늘리고, 수상한 IP 가 어디 것인지 밖에 물어봅니다

오전에 만든 `normalized_logs.json` 을 읽어 룰 ③ 을 더하면 경보가 두 줄에서 세 줄이 됩니다.

[룰 1] 확인 필요: admin - 실패 4회

[룰 2] 의심 IP: 185.220.101.34 - 계정 4개 시도

[룰 3] 심야 접속: 03:17:09 admin (211.45.12.9)

그 IP 를 바깥에 물어 받은 답

185.220.101.34 → Germany · Berlin · 해외 접속

룰 ③ — 새벽에 성공한 로그인을 찾습니다

몇 시부터 야간인지, 몇 회부터 수상한지 — 둘 다 사람이 정해 코드 맨 위에 적어 둡니다.

용어	한 줄 뜻	오늘 쓰는 값
룰 ③ 심야 접속	사람이 자는 시간에 성공한 로그인을 수상하게 본다	INFO accepted 인 줄만 본다
업무 시간	정상이라고 보는 시간대. 몇 시부터 몇 시까지인지는 사람이 정한다	0시~6시를 야간으로 — NIGHT_END = 6

오늘 이 한 줄을 잡아내는 것이 목표입니다

2026-09-29 03:17:09 INFO accepted login for admin from 211.45.12.9

새벽 3시, `admin` 은 네 번 실패한 뒤 결국 성공했습니다

오전 룰 들은 실패만 셧습니다 — 실패는 차단된 접속이고, 성공은 실제로 들어온 접속입니다.

```
2026-09-29 03:11:05 WARN failed login for admin from
211.45.12.9
```

```
2026-09-29 03:12:47 WARN failed login for admin from
211.45.12.9
```

```
... 03:13:58 · 03:15:22 도 같은 실패 (모두 네 번)
```

```
2026-09-29 03:17:09 INFO accepted login for admin from
211.45.12.9
```

↑ 이 한 줄은 아무 룰도 못 잡았다

막힌 실패보다 새벽에 성공한 로그인을 더 의심합니다

룰 ①② — 실패를 썼다

실패가 몰리면 수상하다

그런데 성공한 순간은 안 봤다

admin 은 네 번 실패 뒤 어떻게 됐나

룰 ③ — 성공을 본다

새벽 세 시에 로그인 성공

그 시간에 일하는 사람은 드물다

뚫렸을 수도 있다는 신호

기록의 time 값에서 시(hour)를 숫자로 꺼냅니다

오전 3교시에 쓴 그 방법입니다 — 콜론으로 나눠 첫 값을 꺼내고, `int` 로 숫자로 바꿉니다.

원본 "03:17:09" · 나눈 첫 값 "03" · 숫자로 바꾼 결과 3

```
row = {"time": "03:17:09", "level": "INFO", "user": "admin",  
       "ip": "211.45.12.9"}
```

```
hour = int(row["time"].split(":")[0])
```

```
print(hour)
```

3

를 ③ 의 조건 한 줄에 사람이 정한 것 둘이 들어 있습니다

```
NIGHT_END = 6
```

```
if hour < NIGHT_END and row["level"] == "INFO":
```

`hour < NIGHT_END`

0시부터 6시 전까지를 야간으로 본다 — 사람이 정한 기준

`row["level"] == "INFO"`

성공한 로그인이다(실패는 WARN) — 사람이 정한 관점

`and`

둘 다 참일 때만 참 — 하나라도 거짓이면 건너뛰다

새벽만 보면 새벽의 실패까지, 성공만 보면 낮의 정상 로그인이 전부 걸립니다. 둘을 겹쳐야 한 줄만 남습니다.

룰 ③ 은 새벽이면서 동시에 성공한 기록을 고릅니다

조건 두 개를 and 로 묶습니다 — 새벽이고, 성공이고. 결과는 03시 17분 admin 한 줄입니다.

```
rows = json.load(open("normalized_logs.json", encoding="utf-8"))
NIGHT_END = 6
for row in rows:
    hour = int(row["time"].split(":")[0])
    if hour < NIGHT_END and row["level"] == "INFO":
        print(f"[룰 3] 심야 접속: {row['time']} {row['user']}
        ({row['ip']})")
```

[룰 3] 심야 접속: 03:17:09 admin (211.45.12.9)

세 룰은 같은 기록을 세 각도로 봅니다

룰	무엇을 본다	기준
① 브루트포스	계정별 실패 횟수	3회 이상
② 의심 IP	IP 마다 시도한 계정 수	2개 이상
③ 심야 접속	새벽의 로그인 성공	06시 이전

파일은 한 번만 읽습니다. 읽어 둔 `rows` 를 세 번 돌면 됩니다.

경보를 찍지 말고 `alerts` 리스트 에 담아 둡니다

세 룰의 코드는 그대로입니다 — `print` 를 `append` 로 바꾸는 것 하나뿐입니다.
담아 두면 셀 수 있고, 나중에 파일로 남기거나 메시지로 보낼 수도 있습니다.

지금까지 — 찾자마자 찍었다

```
for user in count:
    if count[user] >= THRESHOLD:
        print(f"[룰 1] 확인 필요: {user}")
```

[룰 1] 확인 필요: admin
— 몇 건인지는 알 수 없다

오늘 — 목록에 담는다

```
alerts = []
for user in count:
    if count[user] >= THRESHOLD:
        alerts.append(f"[룰 1] 확인 필요: {user}")
```

화면에는 아무것도 안 나온다
— `alerts` 에 쌓여 있다

alerts 에 담은 경보를 꺼내 찍고 「경보 3건」 요약을 붙입니다

마지막 줄이 핵심입니다 — 경보가 없으면 「경보 0건」. 아무것도 안 나오는 것과 다릅니다.

```
# 룰 ①②③ 이 append 해 둔 세 줄이 alerts 에 있다
for line in alerts:
    print(line)
print(f"경보 {len(alerts)}건")
```

[룰 1] 확인 필요: admin — 실패 4회

[룰 2] 의심 IP: 185.220.101.34 — 계정 4개 시도

[룰 3] 심야 접속: 03:17:09 admin (211.45.12.9)

경보 3건

5교시에 배운 것을 정리합니다

쓰는 법	뜻
<code>int(time.split(":")[0])</code>	글자 시각에서 시만 숫자로 꺼낸다
<code>hour < NIGHT_END</code>	새벽인지 본다
<code>level == "INFO"</code>	성공한 접속인지 본다
기준값을 맨 위에	바꿀 때 한 곳만 고친다

경보는 파일로 남기지 않습니다.

내일 필요하면 `normalized_logs.json` 을 다시 읽어 룰을 한 번 더 돌립니다.

실습 ④ · 노트북 5교시 절 · 02_0929_pm_rules_api.ipynb

문제 5-1~5-10 · 도전 ★5-6 · ★5-10

사본을 저장하고 맨 위 셀을 실행합니다

①

오전을 못 끝냈어도 이 셀이 `normalized_logs.json` 을 다시 만듭니다

문제 5-1~5-5 · ★5-6 — 룰 ③

②

[룰 3] 심야 접속: 03:17:09 admin 이 나오면 성공

문제 5-7 ~ 5-9 · ★5-10 — 세 룰을 한 번에

③

경보 세 줄과 경보 3건 이 나오면 성공

6교시 낱말 — API · HTTP · 요청과 응답 · HTTP 메서드

용어	한 줄 뜻
API	남의 프로그램에 무엇을 물어볼지 정해 둔 통로
HTTP	그 통로로 오가는 말의 규칙
요청 · 응답	이쪽이 보내는 말 · 저쪽이 돌려주는 말
HTTP 메서드	무엇을 하려는지 밝히는 표시 (GET·POST)

185.220.101.34 가 어느 나라 것인지는 우리 로그에 안 나옵니다

가진 것	없는 것
경보 세 줄 — 누가 언제 무엇을 했나	그 IP 가 정말 위험한 곳인지
로그 파일 안의 사실	바깥의 IP 조회 서비스 — 신고 기록을 모아 둔 곳
남의 컴퓨터를 들여다볼 수는 없다	그래서 물어볼 수 있는 것을 정해 둔 통로가 있다

무엇을 물어볼 수 있는지 저쪽이 미리 정해 공개해 둔 것이 API 입니다.

우리 로그에 없는 정보는 바깥 서비스에 물어봐야 합니다

IP 는 나라와 회사 단위로 지정됩니다 — 그걸 아는 서비스에 물어봐야 합니다.
서버는 접속한 주소만 적으므로 우리 로그에는 그 정보가 없습니다.

[룰 2] 의심 IP: 185.220.101.34 — 계정 4개 시도

이 보고를 받으면 반드시 묻는다

「이 IP, 어디서 온 겁니까?」

우리 로그에는 나라도 소유자도 없다

IP 조회 서비스 — 그 IP 가 어느 나라 · 어느 회사 것인지 알려 주는 곳

IP 는 나라와 회사 단위로 배정되고, 그 기록을 물어보면 알려 주는 서비스가 있습니다.
내일 쓸 곳은 `ipwho.is` — 오늘은 그 응답을 미리 적어 두고 읽습니다.

칸	무엇	이 IP 는
<code>country</code>	어느 나라에 배정된 IP 인가	"Germany"
<code>country_code</code>	나라 이름의 두 글자 표준 코드 — 간편하게 확인가능	"DE" 독일 · "KR" 한국
<code>city</code>	도시 이름	"Berlin"

신고 건수를 주는 IP 평판 조회 서비스도 따로 있지만 이 과정에서는 쓰지 않습니다.

8분 영상 — API는 다른 프로그램에 요청을 보내는 접점입니다

동영상 플레이어 구성 오류

오류 153

YouTube에서 동영상 보기

자세히 알아보기

오류 153

공개된 IP 배정 기록을 조회하면 배정된 기관을 알 수 있습니다

IANA

전 세계 총괄

RIR

대륙별 배정 기관

ISP · 기관

기관에는 ASN이 붙음

회사 · 가정

최종 사용처

185.220.101.34 → 유럽 담당 RIPE NCC → ASN 60729인 기관에 배정

10.1.2.11은 사설 IP 대역이라 공개 배정 대상이 아닙니다 → 조회 결과 `private range`

우리가 주소를 보내면 저쪽이 글자를 돌려줍니다

우리 프로그램

① 요청 — 주소를 보낸다

IP 조회 서비스

들어올 것을 API 로 정해 두었다

우리 프로그램

② 응답 — 글자로 돌아온다

① 보내는 것 — 주소 한 줄이 곧 질문이다

`https://api.example.com/v1/ip/185.220.101.34`

② 돌아오는 것 — JSON 문자열 한 덩어리

`{"ip": "185.220.101.34", "country": "Germany", "city": "Berlin"}`

오가는 것은 주소 한 줄과 글자 한 덩어리, 그것뿐입니다. 가운데 통로가 API 입니다.

HTTP 요청 **하나마다 응답 하나**를 받습니다

요청과 응답은 한 쌍으로 처리됩니다. 오늘의 IP 조회 주소는 한 번에 IP 하나를 묻습니다.

늘 생각할 것	무슨 뜻인가	언제 다루나
답이 안 올 수 있다	주소가 틀렸거나, 권한이 없거나, 저쪽에 오류가 났을 수 있다	7교시 상태코드
IP 가 여러 개면 각각 요청한다	오늘의 조회 주소는 IP 하나씩 받으므로, IP 가 셋이면 요청도 세 번이다	내일 반복문으로 보내기

경보 세 줄에 IP 가 둘이면 조회도 두 번입니다. 같은 IP 가 또 나오면 다시 묻지 않고 앞서 받은 답을 씁니다 — 도전 ☆6-11 이 그 문제입니다.

오늘은 실제 서버로 보내지 않습니다 — 응답을 적어 두고 씁니다

오늘 (9/29) — 읽고 판단

서버가 보냈다고 가정한 응답을 가지고

꺼내 쓰고 · 갈라 판단하는 연습

주소를 만들어 보기까지

내일 (9/30) — 실제로 보내기

`requests` 로 진짜 요청

인터넷 너머에서 받은 응답

키를 코드 밖에 두기

그래서 오늘 코드에는 `requests` 가 한 번도 안 나옵니다.
「왜 인터넷을 안 쓰나요」 소리가 나오면 이 장으로 돌아옵니다.

`json.loads`는 응답의 **JSON 문자열**을 파이썬 딕셔너리로 변환합니다

`text`에 들어 있는 JSON 문자열을 파이썬 딕셔너리로 변환해 `data`에 저장합니다.

`text`는 문자열 · `data`는 딕셔너리 · `data["country"]`는 키로 값을 꺼내는 코드

```
text = '{"country": "Germany", "country_code": "DE", "city":  
"Berlin"}'
```

```
data = json.loads(text)
```

```
print(data["country"], data["city"])
```

```
Germany Berlin
```

json.loads 로 되돌린 data 에서 country_code 를 꺼내 경보를 낼지 정합니다

밖에서 온 값도 우리 로그와 똑같이 다룹니다 — 꺼내고, 기준과 견주고, 판단합니다.
기준 "KR" 도 사람이 정한 것입니다 — 해외 접속을 어디까지 볼지는 회사가 정합니다.

```
text = '{"ip": "185.220.101.34", "country": "Germany", "country_code": "DE"}'  
data = json.loads(text)  
HOME = "KR"  
if data["country_code"] != HOME:  
    print(f"해외 접속: {data['ip']} ({data['country']})")
```

해외 접속: 185.220.101.34 (Germany)

요청을 보낼 주소는 네 조각으로 나뉩니다

```
https://api.example.com/v1/ip/185.220.101.34?full=true
```

조각	무엇
■ 방식	어떻게 주고받을지 — <code>https</code> 는 암호화해서 주고받는다
■ 호스트	어느 서버에게 요청을 보내나 — <code>api.example.com</code>
■ 경로	그 서버의 어느 자리에 — <code>/v1/ip/185.220.101.34</code> . 조회할 IP 가 여기 들어간다
■ 질의(조건을 덧붙이는 자리)	덧붙이는 조건 — ? 뒤가 질의, 여러 개면 & 로 잇는다

`url.split("?")`은 주소를 물음표 앞과 뒤로 나눕니다

```
url = "https://api.example.com/v1/ip/185.220.101.34?full=true"
parts = url.split("?")
print(parts)
```

```
['https://api.example.com/v1/ip/185.220.101.34', 'full=true']
```

`parts[0]`

물음표 앞 · 요청 주소

`parts[1]`

물음표 뒤 · 질의 `full=true`

문제 6-8의 답: `url.split("?")[1] → full=true`

`address.split("/")`은 주소를 슬래시 기준의 리스트로 만듭니다

```
address = "https://api.example.com/v1/ip/185.220.101.34"
parts = address.split("/")
print(parts)
```

```
['https:', '', 'api.example.com', 'v1', 'ip', '185.220.101.34']
```

호스트

```
parts[2]
```

```
api.example.com
```

경로의 마지막 값

```
parts[len(parts) - 1]
```

```
185.220.101.34
```

`https://`에는 슬래시가 연속으로 두 개 있어 인덱스 1에 빈 문자열 `' '`이 생깁니다. 그래서 호스트는 인덱스 2입니다.

HTTP 메서드는 그 주소에 무엇을 하려는지 밝힙니다

메서드	오늘 데이터로 보면	두 번 보내면
GET	IP 정보를 조회한다	같은 조회 결과가 온다
POST	IP 신고를 새로 등록한다	신고가 두 건 생긴다

오늘 우리가 하려는 일은 조회라서 **GET** 입니다. **POST** 는 내일 씁니다.

GET 은 두 번 보내도 결과가 같습니다

네트워크가 느려 응답을 못 받으면 같은 요청을 다시 보내게 됩니다. 그때 무엇을 보냈느냐가 같습니다.

다시 보낸 요청	저쪽에서 일어나는 일	그래서
GET 조회	같은 값을 한 번 더 읽어 온다 — 저쪽 데이터는 그대로다	다시 보내도 안전하다
POST 생성	주문·가입이 한 건 더 만들어진다	사고가 된다

오늘 우리가 하려는 일은 IP 조회라서 GET 입니다 — 실패하면 그냥 다시 보내면 됩니다.

6교시에 배운 것을 정리합니다

쓰는 법	뜻
<code>json.loads(글자)</code>	응답 글자를 딕셔너리로 되돌린다
<code>url.split("?")</code>	주소와 질의를 가른다
<code>url.split("/") [2]</code>	호스트를 꺼낸다
GET / POST	조회한다 / 새로 만든다

오늘은 미리 적어 둔 응답으로 연습합니다. 실제로 보내는 것은 내일 오전입니다.

실습 ⑤ · 노트북 6교시 절

문제 6-1~6-11 · 도전 ★6-7 · ★6-11

문제 6-1~6-6 · ★6-7 — 응답 읽고 판단하기

①

나라 코드를 꺼내고, 국내(KR)가 아니면 경보를 내면 성공

문제 6-8 · 6-9 — 주소 쪼개 읽기

②

질의를 꺼내고, 경로 맨 뒤의 IP 를 꺼내면 성공

문제 6-10 · ★6-11 — 조회 주소 만들기

③

오전 경보의 IP 로 주소를 만들어 목록에 담으면 성공

용어 공부 — 상태코드 · 인증과 인가 · API 키 · 토큰

용어	한 줄 뜻
상태코드	요청이 어떻게 됐는지 알리는 세 자리 숫자
인증 · 인가	누구인지 밝히는 일 · 그 사람에게 권한이 있는지 보는 일
API 키	발급받은 긴 글자를 요청마다 붙이는 방식
토큰	먼저 받아 두고 붙이는, 유효 기간이 있는 글자

답이 안 왔을 때 무엇을 할지는 원인마다 다릅니다

상황	알아야 할 것
요청을 보냈는데 원하는 값이 안 왔다	서버가 죽었나 · 주소가 틀렸나 · 권한이 없나
셋 다 할 일이 다르다	기다린다 / 이쪽에서 고친다 / 사람에게 요청한다
그래서 응답에 세 자리 숫자가 함께 온다	앞자리 하나만 봐도 어느 쪽 잘못인지 갈린다

오늘은 읽는 법까지입니다. 실패를 코드로 다루는 것은 내일 오전에 합니다.

IP 조회 서비스의 처리 결과는 응답에 붙은 **상태코드**로 확인합니다

IP 조회 서비스에 요청하면 주소·인증·서버 문제로 실패할 수 있습니다.

①

주소를 잘못 적었다

— 404 · 주소를 고친다

②

누구인지 안 밝혔다

— 401 · 키나 토큰을 다시 붙인다

③

서버 쪽에서 오류가 났다

— 500 · 잠시 뒤 다시 보낸다

상태코드는 앞자리와 번호를 함께 읽습니다

앞자리	코드	뜻
2로 시작 성공	200	서버가 요청을 정상적으로 처리하고 응답을 보냈다
	201	요청대로 새로 만들었다 — 티켓 생성이 성공하면 이것이 온다
4로 시작 보낸 쪽 잘못	400	요청 모양이 잘못됐다
	401	누구인지 모르겠다
	403	누구인지는 알지만 권한이 없다
	404	그런 주소가 없다
	429	너무 자주 물었다
5로 시작 받는 쪽 잘못	500	서버 안에서 문제가 났다

500은 이쪽 코드를 고쳐도 소용없습니다. 잠시 뒤 다시 보내는 것이 답입니다.

6분 영상 — HTTP 메서드와 상태코드



동영상 플레이어 구성 오류

오류 153

YouTube에서 동영상 보기

자세히 알아보기

오류 153

상태코드 ₂₀₁에서 앞자리 ₂만 꺼내려면 먼저 글자로 바꿉니다

201
숫자 · int

str(201)
글자 "201"로 변환

"201"[0]
첫 글자 "2"

```
code = 201
```

```
# code[0]
```

숫자는 [0]으로 첫 자리를 꺼낼 수 없다

```
text = str(code)
```

```
# 201 → "201"
```

```
first = text[0]
```

```
# "201" → "2"
```

[0]은 첫 번째 글자를 꺼내는 문법입니다. 그래서 숫자를 문자열로 바꾼 뒤 사용합니다.

꺼낸 앞자리는 숫자가 아니라 글자라서 따옴표를 붙여 비교합니다

`str()` 로 바꿨으니 글자끼리 비교해야 합니다.

따옴표를 빼고 `== 2` 라고 쓰면 오류가 안 나고 그냥 거짓이 되어 조건이 영원히 안 맞습니다.

```
code = 201
first = str(code)[0]
if first == "2":           # 따옴표를 붙여 비교한다
    print("성공")
```

성공

앞자리 2·4·5 마다 할 일을 `elif` 로 세 갈래로 가릅니다

`elif` 는 처음 참이 된 갈래 하나만 실행됩니다.

할 일이 셋 다 다릅니다 — 성공이면 쓰고, 4 면 내 요청을 고치고, 5 면 기다렸다 다시 보냅니다.

```
code = 403
first = str(code)[0]
if first == "2":
    print("성공")
elif first == "4":
    print("내가 보낸 요청이 잘못됐다")
else:
    print("서버 쪽 문제다")
```

앞자리가 4 인 응답이 몇 건인지 셉니다

세는 방법은 오전에 실패를 센 것과 같고 건주는 값만 앞자리로 바뀌었습니다.

4 로 시작하는 응답이 갑자기 늘면 이쪽 코드나 API 키에 문제가 생긴 것입니다.

```
codes = [200, 404, 500, 403, 200, 401]
```

```
bad = 0
```

```
for code in codes:
```

```
    if str(code)[0] == "4":
```

```
        bad = bad + 1
```

```
print("보낸 쪽 잘못", bad, "건")
```

보낸 쪽 잘못 3건

`str()` 과 `[0]` 으로 상태코드의 앞자리를 꺼냅니다

문자열도 정수 인덱스로 문자를 꺼냅니다 — `[0]`은 첫 글자입니다.

```
code = 403
first = str(code)[0]
if first == "2":
    print("성공")
elif first == "4":
    print("내가 보낸 요청이 잘못됐다")
else:
    print("서버 쪽 문제다")
```

내가 보낸 요청이 잘못됐다

200은 요청 처리가 성공했다는 뜻이지, 조회 결과가 있다는 뜻은 아닙니다

```
status = 200
```

```
data = {"success": False, "message": "private range"}
```

상태코드 200

IP 조회 서비스가
요청을 받고 응답함

success: false

10.1.2.11은 사설 IP라
공개된 소유 정보가 없음

통신은 성공했지만 조회할 정보는 없습니다. 상태코드와 응답 내용은 서로 다른 결과입니다.

실무 코드는 상태코드와 응답 내용을 모두 확인합니다

```
if status == 200 and data["success"]:  
    print(data["country"])  
else:  
    print(data["message"])
```

private range

상태코드

HTTP 공통 규칙

200 · 401 · 500

응답 내용의 필드

서비스가 정한 형식

success · message

상태코드의 뜻은 공통이지만, 응답 필드 이름은 API 문서에서 확인합니다.

401 과 403의 차이점

비교하는 점	401	403
서버가 하는 말	당신이 누구인지 모르겠다	누구인지 알겠는데 안 된다
어디서 막혔나	인증	인가 (권한 없음)
할 일	토큰을 다시 받아 붙인다	담당자에게 권한을 요청한다

둘 다 앞자리 4 — 서버가 요청을 보낸 우리 쪽에서 확인할 문제라고 알리는 코드입니다. 다만 해결 방법은 완전히 다릅니다.

우리 프로그램이 IP 조회 서비스에 자신을 인증하는 방식은 API 키와 토큰 두 가지입니다

방식	어떻게	약점
API 키	발급받은 긴 글자를 요청마다 붙인다	한 번 새면 바꾸기 전까지 계속 쓰인다
토큰	먼저 받아 두고 요청마다 붙인다	유효 기간이 있어 만료되면 다시 받는다

API 키나 토큰을 붙이는 자리는 요청의 헤더입니다 — **Authorization: Bearer** 토큰 꿀. 키를 코드 밖에 두는 방법은 내일 배웁니다.

상태코드와 **할 일**을 딕셔너리로 짝지어 둡니다

상태코드가 열 가지로 늘어도 `if` 를 열 번 쓰지 않습니다.

코드를 키로, 사람이 해야 할 일을 값으로 적어 두면 같은 표를 계속 꺼내 쓸 수 있습니다.

```
todo = {401: "토큰을 다시 받는다",  
        403: "권한을 요청한다",  
        404: "주소를 고친다",  
        500: "잠시 뒤 다시 보낸다"}
```

```
code = 403
```

```
print(code, todo[code])
```

403 권한을 요청한다

대응표에 없는 상태코드가 와도 멈추지 않게 합니다

대응표에 없는 코드가 와도 `if code in meaning:` 으로 먼저 확인해 멈추지 않게 하고 나머지는 `else`로 처리합니다

```
meaning = {200: "처리했다", 404: "그런 주소가 없다"}
```

```
for code in [200, 404, 999]:  
    if code in meaning:  
        print(code, meaning[code])  
    else:  
        print(code, "알 수 없음")
```

200 처리했다

404 그런 주소가 없다

999 알 수 없음

토큰을 `Authorization` 헤더 값으로 설정합니다

실수가 나는 자리는 "`Bearer` " 뒤의 공백 한 개입니다 — 빼면 `Bearerabc123` 이 되어 서버가 토큰을 읽지 못합니다.

오늘은 헤더를 만들기만 하고, 요청에 붙여 보내는 것은 내일 합니다.

```
token = "abc123"
headers = {"Authorization": "Bearer " + token}
print(headers)
```

```
{'Authorization': 'Bearer abc123'}
```

7교시에 배운 것을 정리합니다

쓰는 법	뜻
<code>str(code)[0]</code>	상태코드 앞자리
<code>2 / 4 / 5</code>	성공 / 보낸 쪽 잘못 / 서버 잘못
<code>401 / 403</code>	누구인지 모르겠다 / 알겠는데 안 된다
<code>{"Authorization": "Bearer " + token}</code>	요청에 붙이는 헤더

앞자리 하나로 고칠 쪽이 이쪽인지 저쪽인지 갈립니다.
인증과 인가가 10월 하순 접근통제 과목의 첫 장입니다.

실습 ⑥ · 노트북 7교시 절

문제 7-1~7-11 · 도전 ★7-7 · ★7-11

문제 7-1~7-6 · ★7-7 — 상태코드

- ① 앞자리로 세 갈래를 나누고
200 인데 실패
인 것도 가려내면 성공

문제 7-8 — 401 ↔ 403

- ② 막힌 이유에 따라 할 일을 다르게 내면 성공

문제 7-9 · 7-10 · ★7-11 — 헤더와 재시도

- ③ 헤더를 만들고, 401 이면 한 번만 다시 보내면 성공

상태코드부터 토큰까지, 다섯 용어를 정리합니다

용어 (한글 발음)	한 줄 뜻	비유
상태코드 status code · 스테이터스 코드	요청이 어떻게 됐는지 알리는 세 자리 숫자	접수 도장
인증 authentication · 어센티케이션	누구인지 확인하는 일	신분증 확인
인가 authorization · 어소라이제이션	그 사람이 이 일을 해도 되는지 확인	출입 허가
API 키 key · 키	발급받은 긴 비밀 문자열	출입 카드
토큰 token · 토큰	일정 시간만 쓰는 임시 열쇠	기간 정해진 방문증

드라이브에 오늘 파일이 다섯 개 남았는지 확인합니다

무엇	어디에	언제
<code>normalize_logs.py</code> (정규식 판)	드라이브 <code>agent_core</code>	오전 3교시
<code>normalized_logs.json</code> · <code>unmatched_logs.txt</code>	드라이브 <code>agent_core</code>	오전 3교시
탐지 룰 세 개 · 경고 세 줄	화면에만	오전 4교시 · 오후 5교시

경보를 파일로 안 남긴 이유는 하나입니다 — 교안에 없는 이름을 새로 짓지 않습니다.

8교시에는 새것 없이 오늘 만든 것을 마무리합니다

어디	도전 문제
오전	★2-6·★2-10 (값 세기) · ★3-5·★3-8·★3-12 (그룹) · ★4-6·★4-10 (룰 ①②)
5교시	★5-6 기준 시각 정하기 · ★5-10 두 룰이 같이 지목한 것
6·7교시	★6-7·★6-11 (응답 세 개·중복 조회) · ★7-7·★7-11 (앞자리·조회 붙이기)

기본 문제를 못 끝낸 분은 도전이 아니라 기본부터 합니다. 하나라도 더 해 보는 것이 목적입니다.

과제 — 상태코드를 직접 받아 보고 왜 숫자인지 알아봅시다

과제	무엇을 한다
① 직접 받아 보기 화면 사진 2장	F12 → Network 탭에서 새로고침. 줄마다 Status 에 숫자가 붙습니다. 없는 주소 (naver.com/abcdefg)로 404 도 만들어 봅니다
② 다섯 개 정리 다섯 줄 표	200 401 403 404 500 — 뜻 한 줄 + 받으면 내가 할 행동 한 줄. 수업 표현을 베끼지 말고 자기 문장으로
③ 왜 숫자인가 내 답 3~4줄	「서버가 그냥 한글 설명을 보내면 안 되나?」 서버마다 말이 다르면 프로그램이 무엇을 못 하는지 각자 찾아봅니다

노트북 맨 아래 텍스트 셀에 적고, 내일 아침 30분에 깃허브에 같이 올립니다.

오늘 배운 것을 한눈에 봅니다

오전 — 줄글에서 값을 꺼내 판단한다

`split` 결과의 인덱스가 조용히 달라진다

글자 모양으로 찾기 · 그룹 · `.groupdict()`

안 맞는 줄을 파일로 남긴다 · 탐지 룰 ①②

오후 — IP 조회 서비스에 물어본다

룰 ③ 심야 성공 · 경보 세 줄

요청과 응답 · 주소 네 조각 · HTTP 메서드

상태코드 · 401↔403 · 인증 두 방식

내일 9/30 오전은 `requests` — 오늘 미리 적어 두었던 응답을 실제로 받아 옵니다.

오후는 최주용 강사입니다.

내일 아침에 필요한 셋 — 깃허브 계정 · 드라이브 파일 · 설치할 것

①

깃허브 계정

— 어제 낸 숙제입니다. 내일 아침 30분에 깃허브 **쓰니다**

②

드라이브 `agent_core`

— 오늘 만든 파일 셋이 들어 있어야 합니다

③

깃배시

— 어제 깐 그대로면 됩니다. 새로 깔 것은 없습니다



17:30 — 여기서 수업을 마치고 오후 리추얼로 넘어갑니다.

리추얼 20분(17:30~17:50) 뒤 **17:50** 에 끝납니다. 늦게 끝내지 않습니다.